

Język Python

Wykład 03

Fizyka Medyczna

20 października 2026

Podstawy funkcji

Jacek Gatlik

e-mail: jacek.gatlik@agh.edu.pl
www: https://galaxy.agh.edu.pl/~jgatlik/

Wydział Fizyki i Informatyki Stosowanej,
Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie

Plan wykładu

Podsumowanie podstawowych typów danych

Podstawy funkcji

Zadanie dodatkowe

Sekcja 1

Podsumowanie podstawowych typów danych

Łańcuchy znakowe (Strings)

- **Definicja:** Sekwencje znaków używane do reprezentacji tekstu.
- **Mutowalność:** ***Niemutowalne*** – po utworzeniu nie można zmienić ich zawartości.
- **Indeksowanie i Slicing**: Umożliwiają dostęp do poszczególnych znaków lub podłańcuchów za pomocą indeksów i operatora `[:]`.
- Operacje:
  - Konkatenacja: Łączenie dwóch łańcuchów za pomocą operatora +.
  - Powielanie: Powtarzanie łańcucha za pomocą operatora *.
  - Metody wbudowane: `split()`, `join()`, `replace()`, `find()`, `upper()`, `lower()`.
- **Przykład:**

```
s = "Python"
print(s[0])    # Output: 'P'
print(s[1:4])  # Output: 'yth'
```

Listy (Lists)

- **Definicja:** Sekwencje elementów dowolnego typu (**mutowalne**).
- **Mutowalność:** ***Mutowalne*** – można zmieniać ich zawartość po utworzeniu.
- **Indeksowanie i Slicing**: Dostęp do elementów listy za pomocą indeksów oraz do jej fragmentów za pomocą operatora `[:]`.
- Operacje:
  - Dodawanie elementów: `append()`, `insert()`, `extend()`.
  - Usuwanie elementów: `remove()`, `pop()`, `clear()`.
  - Sortowanie i odwracanie: `sort()`, `reverse()`.
- **Przykład:**

```
lst = [1, 2, 3]
lst.append(4)       # lst: [1, 2, 3, 4]
lst[0] = 0          # lst: [0, 2, 3, 4]
```

Krotki (Tuples)

- **Definicja:** Sekwencje elementów dowolnego typu (**niemutowalne**).
- **Mutowalność:** ***Niemutowalne*** – nie można podmienić elementów krotki; zawarta w niej lista nadal może się zmieniać.
- **Indeksowanie i Slicing**: Dostęp do elementów krotki za pomocą indeksów oraz do jej fragmentów za pomocą operatora `[:]`.
- Operacje:
  - Ograniczone do operacji niemodyfikujących: dostęp do elementów, iteracja, zliczanie.
- **Przykład:**

```
tup = (1, 2, 3)
print(tup[1])   # Output: 2
tup[0] = 0    # Błąd: krotki są niemutowalne
```

Słowniki (Dictionaries)

- **Definicja:** Kolekcje par klucz-wartość, gdzie klucze są **unikalne i hashowalne** (np. napisy, liczby; nie listy).
- **Mutowalność:** ***Mutowalne*** – można dodawać, usuwać i modyfikować pary klucz-wartość.
- **Indeksowanie**: Dostęp do wartości poprzez klucze, nie indeksy.
- Operacje:
  - Dodawanie/aktualizacja: `dict[key] = value`.
  - Usuwanie: `del dict[key]`, `pop()`, `clear()`.
  - Metody wbudowane: `keys()`, `values()`, `items()`, `get()`, `update()`.
- **Przykład:**

```
dct = {'a': 1, 'b': 2}
print(dct['a'])       # Output: 1
dct['c'] = 3          # dct: {'a': 1, 'b': 2, 'c': 3}
```

Zbiory (Sets)

- **Definicja: Nieuporządkowane** kolekcje **unikalnych, hashowalnych** elementów.
- **Mutowalność:** ***Mutowalne*** – ale istnieją również niemutowalne zbiory (`frozenset`).
- **Indeksowanie**: **Brak** – elementy są nieuporządkowane.
- Operacje:
  - Teoria zbiorów: `union()`, `intersection()`, `difference()`, `symmetric_difference()`.
  - Dodawanie/usuwanie: `add()`, `remove()`, `discard()`, `pop()`.
- **Przykład:**

```python
st = {1, 2, 3}
st.add(4)              # st: {1, 2, 3, 4}
st.remove(2)           # st: {1, 3, 4}
```

Porównanie kluczowych cech

Cechy	Łańcuchy znakowe	Listy	Krotki	Słowniki	Zbiory
Mutowalność	Nie	Tak	Nie	Tak	Tak
Uporządkowanie	Tak	Tak	Tak	Tak*	Nie
Indeksowanie	Tak	Tak	Tak	Klucze	Nie
Duplikaty	Tak	Tak	Tak	Klucze nie	Nie
Typ elementów	Znaki	Dowolne	Dowolne	Klucz: hashowalny Wartość: dowolna	Hashowalne
Operacje specjalne	Operacje na tekście	Metody listy	-	Operacje na kluczach i wartościach	Operacje teorii zbiorów

Podobieństwa i różnice

Podobieństwa:

- **Iterowalność**: Wszystkie typy są iterowalne za pomocą pętli `for`.
- **Zapis literałów**:
  - `''` lub `""` dla łańcuchów znakowych.
  - `[]` dla list.
  - Przecinek tworzy krotkę: `(1,)`; samo `(1)` to liczba. Pusta krotka: `()`.
  - `{}` dla słowników i zbiorów (puste `{}` tworzy słownik, `set()` tworzy pusty zbiór).

Podobieństwa i różnice

Różnice:

- **Mutowalność**:
  - **Mutowalne**: Listy, słowniki, zbiory.
  - **Niemutowalne**: Łańcuchy znakowe, krotki.
- **Uporządkowanie**:
  - **Uporządkowane**: Łańcuchy znakowe, listy, krotki, słowniki (od Python 3.7).
  - **Nieuporządkowane**: Zbiory.
- **Dostęp do elementów**:
  - **Indeksowanie numeryczne**: Łańcuchy znakowe, listy, krotki.
  - **Klucze**: Słowniki.
  - **Brak indeksowania**: Zbiory.
- **Unikalność elementów**:
  - **Wymagana**: Zbiory (elementy), słowniki (klucze).
  - **Niewymagana**: Łańcuchy znakowe, listy, krotki.

Zastosowania praktyczne

- **Łańcuchy znakowe**: Przetwarzanie tekstu, komunikacja z użytkownikiem, parsowanie danych.
- **Listy**: Kolejki, stosy, dynamiczne tablice, przechowywanie sekwencji danych z możliwością modyfikacji.
- **Krotki**: Stałe zestawy danych, zwracanie wielu wartości z funkcji, klucze w słownikach (jeśli elementy są hashowalne).
- **Słowniki**: Mapowanie kluczy na wartości, szybki dostęp do danych na podstawie klucza, reprezentacja obiektów.
- **Zbiory**: Operacje matematyczne na zbiorach, eliminacja duplikatów, testowanie przynależności.

Sekcja 2

Podstawy funkcji

Dlaczego używamy funkcji?

Funkcje są fundamentalnym elementem programowania, umożliwiającym organizację kodu w sposób modularny i zrozumiały. Oto główne powody, dla których używamy funkcji:

- **Reużywalność kodu**: Funkcje pozwalają na wielokrotne wykorzystanie tego samego kodu bez konieczności jego powielania. Ułatwia to utrzymanie i aktualizację kodu.
- **Modularność:** Dzieląc kod na mniejsze, logiczne części, zwiększamy jego czytelność i ułatwiamy zarządzanie skomplikowanymi projektami.
- **Abstrakcja:** Funkcje umożliwiają ukrycie szczegółów implementacji, prezentując jedynie interfejs (nazwę funkcji i jej parametry), co ułatwia zrozumienie kodu na wyższym poziomie.
- **Testowalność:** Mniejsze, izolowane fragmenty kodu są łatwiejsze do testowania jednostkowego, co przyczynia się do poprawy jakości oprogramowania.
- **Ułatwienie debugowania:** Gdy kod jest podzielony na funkcje, łatwiej jest zlokalizować i naprawić błędy.

Definicja funkcji

W Pythonie funkcja jest zdefiniowanym blokiem kodu, który wykonuje określone zadanie i może zwracać wartość. Definicja funkcji składa się z następujących elementów:

- Słowo kluczowe ***def***: Służy do zdefiniowania nowej funkcji.
- Nazwa funkcji: Powinna być opisowa i zgodna z konwencjami nazewnictwa PEP 8 (małe litery, ewentualnie podkreślenia).
- Lista parametrów: W nawiasach okrągłych, określa dane wejściowe funkcji.
- Dwukropek ***:***: Kończy nagłówek definicji funkcji.
- Blok kodu: Wcięty kod, który jest ciałem funkcji.
- Opcjonalna instrukcja ***return***: Służy do zwrócenia wartości z funkcji.

```
1 def dodaj(a, b):
2     wynik = a + b
3     return wynik
```

Parametry to nazwy `a`, `b` w definicji; **argumenty** to wartości przekazane przy wywołaniu, np. `5`, `7`.
Aby użyć funkcji, należy ją wywołać, podając wymagane argumenty:

```
1 suma = dodaj(5, 7)
2 print(suma)  # Wyświetli: 12
```

return a print

- `def` tworzy funkcję; jej ciało wykonuje się dopiero przy wywołaniu.
- `return` kończy wywołanie i przekazuje wynik do dalszych obliczeń.
- `print()` wyświetla tekst. Samo wypisanie wyniku nie zwraca go z funkcji.
- Dojście do końca funkcji lub samo `return` zwraca `None`.

```python
def dodaj(a, b):
    return a + b

def wypisz_sume(a, b):
    print(a + b)

wynik = dodaj(3, 4)
print(wynik * 2)               # 14
brak_wyniku = wypisz_sume(3, 4) # wyświetla 7
print(brak_wyniku)             # None
```

Argumenty domyślne

Argumenty domyślne pozwalają na zdefiniowanie wartości, które zostaną użyte, jeśli podczas wywołania funkcji nie zostaną przekazane odpowiednie argumenty.

```
def nazwa_funkcji(parametr1=1, parametr2=2):
    pass  # miejsce na ciało funkcji
```

Zasady:

- Wśród parametrów pozycyjnych parametry z wartościami domyślnymi muszą występować po parametrach wymaganych.
- Podczas wywoływania funkcji można pominąć argumenty z wartościami domyślnymi.

Przykład:

```
def powitanie(imie, jezyk='pl'):
    if jezyk == 'pl':
        print(f'Cześć, {imie}!')
    elif jezyk == 'en':
        print(f'Hello, {imie}!')
    else:
        print(f'Witaj, {imie}!')
powitanie('Anna')          # Użyje domyślnego języka 'pl'
powitanie('John', 'en')    # Przekaże język 'en'
```

Mutowalne wartości domyślne

Wartości domyślne są obliczane tylko raz, w momencie definicji funkcji. Dlatego nie zaleca się używania jako wartości domyślnych obiektów mutowalnych, takich jak listy czy słowniki.
Przykład problemu z mutowalnymi wartościami domyślnymi:

```
def dodaj_element(element, lista=[]):
    lista.append(element)
    return lista

print(dodaj_element(1))  # Wynik: [1]
print(dodaj_element(2))  # Wynik: [1, 2] - lista jest współdzielona!
```

Aby uniknąć tego problemu, należy użyć `None` jako wartości domyślnej:

```
def dodaj_element(element, lista=None):
    if lista is None:
        lista = []
    lista.append(element)
    return lista
```

Argumenty wariadyczne (zmienna liczba argumentów)

Argumenty wariadyczne pozwalają na przekazanie do funkcji dowolnej liczby argumentów pozycyjnych. W Pythonie realizuje się to za pomocą operatora *.

```
def nazwa_funkcji(*args):
    pass  # miejsce na ciało funkcji
```

args** to konwencjonalna nazwa, gdzie ***args jest krotką zawierającą wszystkie dodatkowe argumenty pozycyjne.

```
def suma(*liczby):
    wynik = 0
    for liczba in liczby:
        wynik += liczba
    return wynik

print(suma(1, 2, 3))       # Wynik: 6
print(suma(4, 5, 6, 7))    # Wynik: 22
```

Działanie:

- `*args` zbiera dodatkowe argumenty **pozycyjne**, których nie przypisano wcześniejszym parametrom; nie zbiera argumentów nazwanych.
- Pozwala to na pisanie funkcji, które są bardziej elastyczne i mogą obsługiwać różne liczby argumentów.

Argumenty wariadyczne (zmienna liczba argumentów)

Zastosowania:

- Gdy nie wiadomo z góry, ile argumentów będzie potrzebnych.
- Przy tworzeniu funkcji matematycznych, np. `min`, `max`, które działają na nieokreślonej liczbie argumentów.

Łączenie z argumentami nazwanymi:
W typowej definicji funkcji stosujemy następującą kolejność parametrów:

1. Parametry pozycyjne lub nazwane: najpierw wymagane, potem z wartościami domyślnymi.
2. `*args`: dodatkowe argumenty pozycyjne.
3. Parametry tylko nazwane: wymagane lub z wartościami domyślnymi, w dowolnej kolejności.
4. `**kwargs`: dodatkowe argumenty nazwane.

Argumenty wariadyczne (zmienna liczba argumentów)

Przykład z łączeniem różnych typów argumentów:

```
def funkcja(a, b=2, *args):
    print(f'a = {a}')
    print(f'b = {b}')
    print(f'args = {args}')
funkcja(1)                    # a=1, b=2, args=()
funkcja(1, 3, 4, 5)           # a=1, b=3, args=(4, 5)
```

Rozpakowywanie argumentów:
Operatora * można również użyć do rozpakowywania sekwencji podczas wywoływania funkcji.

```
def mnozenie(a, b, c):
    return a * b * c
liczby = (2, 3, 4)
print(mnozenie(*liczby))  # Wynik: 24
```

- Należy pamiętać o właściwej kolejności argumentów w definicji funkcji.
- Operator * pełni podwójną rolę: zbierania argumentów w krotkę i rozpakowywania sekwencji do argumentów funkcji

Argumenty słów kluczowych

Argumenty słów kluczowych (ang. keyword arguments) to sposób przekazywania argumentów do funkcji, w którym podajemy nazwę parametru wraz z jego wartością. Pozwala to na zwiększenie czytelności kodu oraz elastyczność w wywoływaniu funkcji, ponieważ kolejność argumentów przestaje mieć znaczenie.

```
1 def przedstaw_sie(imie, wiek):
2     print(f"Nazywam się {imie} i mam {wiek} lat.")
3
4 przedstaw_sie('Anna', 30) # Wywołanie z argumentami pozycyjnymi
5 przedstaw_sie(wiek=25, imie='Jan') # Wywołanie z argumentami słów kluczowych
```

Zalety argumentów słów kluczowych

- Czytelność: Dzięki użyciu nazw parametrów kod staje się bardziej zrozumiały.
- Elastyczność: Możliwość zmiany kolejności argumentów podczas wywołania funkcji.
- Domyślne wartości: W połączeniu z argumentami domyślnymi pozwala na tworzenie funkcji z opcjonalnymi parametrami.

Łączenie z argumentami pozycyjnymi

Można łączyć argumenty pozycyjne z argumentami słów kluczowych w jednym wywołaniu funkcji. Ważne jest jednak, aby zwykłe argumenty pozycyjne były podane przed argumentami słów kluczowych.

Argumenty słów kluczowych

```python
def zamowienie(danie, napoj, deser):
    print(f"Zamówiono: {danie}, {napoj}, {deser}.")

zamowienie('Pizza', napoj='Cola', deser='Lody')
```

Ograniczenia:

- Zwykłe argumenty pozycyjne muszą być podane przed argumentami słów kluczowych.
- Nazwy muszą odpowiadać parametrom funkcji, chyba że przyjmuje ona dodatkowe nazwy przez `**kwargs`.

Błędne wywołanie:

```python
# Spowoduje błąd, ponieważ argument pozycyjny podano po słowie kluczowym
zamowienie(danie='Spaghetti', 'Sok', deser='Ciasto')
```

Praktyczne wskazówki:

- Używaj argumentów słów kluczowych, gdy funkcja ma wiele parametrów lub gdy chcesz zwiększyć czytelność kodu.
- W dokumentacji funkcji zawsze podawaj nazwy parametrów i ich oczekiwane typy.

Wariadyczne argumenty słów kluczowych

Wariadyczne argumenty słów kluczowych pozwalają na przekazanie do funkcji dowolnej liczby argumentów nazwanych, które nie zostały zdefiniowane w nagłówku funkcji. W Pythonie realizuje się to za pomocą operatora **.

```
def funkcja(**kwargs):
    pass  # miejsce na ciało funkcji
```

- ***kwargs*** (od keyword arguments) to konwencjonalna nazwa, która jest słownikiem zawierającym wszystkie dodatkowe argumenty nazwane.

```
def informacje_o_produktach(**produkty):
    for nazwa, cena in produkty.items():
        print(f"Produkt: {nazwa}, Cena: {cena} PLN")
informacje_o_produktach(Chleb=3.5, Masło=7.99, Mleko=2.99)
```

```
Produkt: Chleb, Cena: 3.5 PLN
Produkt: Masło, Cena: 7.99 PLN
Produkt: Mleko, Cena: 2.99 PLN
```

Wariadyczne argumenty słów kluczowych

Zastosowania:

- Gdy funkcja musi obsłużyć nieznaną liczbę argumentów nazwanych.
- Tworzenie funkcji z elastycznym interfejsem, np. rejestrowanie zdarzeń z różnymi parametrami.

Łączenie z innymi argumentami

Można łączyć wariadyczne argumenty słów kluczowych z argumentami pozycyjnymi i `*args`.

```
1 def funkcja(a, b, *args, **kwargs):
2     print(f"a = {a}")
3     print(f"b = {b}")
4     print(f"args = {args}")
5     print(f"kwargs = {kwargs}")
6
7 funkcja(1, 2, 3, 4, x=5, y=6)
```

```
a = 1
b = 2
args = (3, 4)
kwargs = {'x': 5, 'y': 6}
```

Wariadyczne argumenty słów kluczowych

Rozpakowywanie słowników

Operator ** może być użyty do rozpakowywania słowników podczas wywoływania funkcji.

```python
def ustawienia_uzytkownika(imie, wiek, miasto):
    print(f"Imię: {imie}, Wiek: {wiek}, Miasto: {miasto}")

dane = {'imie': 'Ewa', 'wiek': 27, 'miasto': 'Gdańsk'}
ustawienia_uzytkownika(**dane)
```

Uwaga na kolizje nazw

Każdy parametr może otrzymać wartość tylko raz. Przekazanie jej jednocześnie pozycyjnie i po nazwie powoduje `TypeError`.

```python
def funkcja(a, **kwargs):
    print(f"a = {a}")
    print(f"kwargs = {kwargs}")

# Spowoduje TypeError: funkcja() got multiple values for argument 'a'
funkcja(1, a=2)
```

Funkcje akceptujące wszystkie dane wejściowe

Funkcje akceptujące wszystkie dane wejściowe są w stanie przyjąć dowolną liczbę argumentów zarówno pozycyjnych, jak i nazwanych. Jest to osiągane poprzez użycie zarówno `*args`, jak i `**kwargs` w definicji funkcji.

```
def funkcja(*args, **kwargs):
    pass
```

Przykład:

```
def loguj_zdarzenie(*args, **kwargs):
    print("Zdarzenie:")
    for arg in args:
        print(f" - {arg}")
    for klucz, wartosc in kwargs.items():
        print(f" * {klucz}: {wartosc}")
loguj_zdarzenie('Błąd systemu', kod=500, krytyczny=True)
```

```
Zdarzenie:
- Błąd systemu
* kod: 500
* krytyczny: True
```

Kolejność argumentów w definicji funkcji - przykład

1. Parametry pozycyjne lub nazwane: wymagane, potem z wartościami domyślnymi.
2. `*args`: dodatkowe argumenty pozycyjne.
3. Parametry tylko nazwane (wymagane i domyślne w dowolnej kolejności).
4. `**kwargs`: dodatkowe argumenty nazwane.

Po `*args` parametry podajemy tylko po nazwie. Jeśli nie zbieramy dodatkowych argumentów pozycyjnych, wystarczy samo `*`, np. `def f(a, *, c):`.

```
1 def funkcja(a, b=2, *args, c, d=4, **kwargs):
2     print(f"a = {a}")
3     print(f"b = {b}")
4     print(f"args = {args}")
5     print(f"c = {c}")
6     print(f"d = {d}")
7     print(f"kwargs = {kwargs}")
8
9 funkcja(1, 3, 5, 6, c=7, e=8)
```

Łączenie i rozpakowywanie argumentów

```python
def funkcja(a, b=2, *args, c, d=4, **kwargs):
    print(f"a = {a}")
    print(f"b = {b}")
    print(f"args = {args}")
    print(f"c = {c}")
    print(f"d = {d}")
    print(f"kwargs = {kwargs}")

funkcja(1, 3, 5, 6, c=7, e=8)
```

```
a = 1
b = 3
args = (5, 6)
c = 7
d = 4
kwargs = {'e': 8}
```

Przekazywanie parametrów

Parametr funkcji jest lokalną nazwą wskazującą przekazany obiekt. Samo przekazanie argumentu nie tworzy kopii obiektu.

- **Modyfikacja obiektu** (np. `lista.append(...)`): Jest widoczna przez wszystkie nazwy wskazujące ten sam obiekt.
- **Przypisanie parametrowi** (np. `a = 10`, `lista = []`): Zmienia tylko lokalne powiązanie nazwy. Obiektów niemutowalnych nie można zmieniać w miejscu.

```python
def zmien_wartosc(a):
    a = 10
x = 5
zmien_wartosc(x)
print(x)  # Wyświetli: 5
```

```python
def dodaj_element(lista):
    lista.append(4)
moja_lista = [1, 2, 3]
dodaj_element(moja_lista)
print(moja_lista)  # Wyświetli: [1, 2, 3, 4]
```

Zwracane wartości

Instrukcja `return`

`return` kończy bieżące wywołanie funkcji i zwraca wartość. Samo `return` lub dojście do końca funkcji bez wykonania `return` z wartością daje `None`.

```
def dodaj(a, b):
    return a + b

wynik = dodaj(3, 4)
print(wynik)  # Wyświetli: 7
```

Zwracanie wielu wartości

Zapis `return suma, roznica` zwraca jeden obiekt: krotkę, którą można rozpakować do kilku zmiennych.

```
def operacje(a, b):
    suma = a + b
    roznica = a - b
    return suma, roznica

s, r = operacje(5, 2)
print(s)  # Wyświetli: 7
print(r)  # Wyświetli: 3
```

Zwracane wartości - błędy

Zwracanie błędów w funkcjach

Funkcje mogą sygnalizować wystąpienie błędu na różne sposoby:

1. Zwracanie specjalnej wartości: Na przykład `None` lub wartość `sentinel`.

```python
def znajdz_element(lista, element):
    if element in lista:
        return lista.index(element)
    else:
        return -1  # Wartość sentinel
```

2. Podnoszenie wyjątku: Używając instrukcji `raise`.

```python
def dzielenie(a, b):
    if b == 0:
        raise ValueError("Dzielenie przez zero!")
    return a / b
```

Kiedy zwracać błąd, a kiedy podnosić wyjątek?

- **Specjalna wartość:** Gdy brak wyniku jest normalną sytuacją; należy ją sprawdzić przed użyciem (`-1` to także poprawny indeks ostatniego elementu).
- **Wyjątek:** Gdy funkcja nie może wykonać operacji zgodnie z umową, np. z powodu błędnego argumentu; wyjątek nie musi oznaczać awarii programu.

Sekcja 3

Zadanie dodatkowe

Zadanie dodatkowe

Zadanie (2 punkty)

Proszę napisać program szyfrujący oraz deszyfrujący wiadomości z wykorzystaniem szyfru Vigenère'a. Program powinien zawierać dwie funkcje:

- `vigenere_encrypt(plaintext, key)` zamienia tekst na wielkie litery, ignoruje spacje, a następnie szyfruje każdą literę.
- `vigenere_decrypt(encryptedtext, key)` deszyfruje tekst; zwraca wielkie litery bez usuniętych wcześniej spacji.

Przyjmij alfabet `A-Z` (bez polskich znaków). Klucz po zamianie na wielkie litery musi być niepusty i zawierać tylko `A-Z`. W tekście pomijaj spacje, a inne znaki spoza alfabetu odrzucaj przez `ValueError`.

Szyfr Vigenère'a
Jest to szyfr podstawieniowy, w którym przesunięcie zależy od kolejnych liter powtarzanego klucza.
Zasada działania:

1. Klucz jest sekwencją liter, która wskazuje o ile miejsc należy przesunąć każdą literę w tekście.
2. Klucz jest „powtarzany" tak, aby jego długość pasowała do długości szyfrowanej wiadomości.
3. Przesuwamy literę wiadomości o wartość litery klucza: „A" oznacza 0, „B" oznacza 1, „C" oznacza 2 itd.

Zadanie dodatkowe

Szyfrowanie:
Każdą literę tekstu zamieniamy na liczbę odpowiadającą jej pozycji w alfabecie, a następnie dodajemy wartość odpowiadającą literze w kluczu (również zamienioną na liczbę). Wynik konwertujemy z powrotem na literę.
Przykład:
Tekst: `"HELLO"`
Klucz: `"KEY"`
Powtórzony klucz: `"KEYKE"`. Każda litera tekstu zostaje przesunięta o odpowiednią wartość wynikającą z klucza.

- `H + K = (7 + 10) % 26 = 17` $\rightarrow$ `R`
- `E + E = (4 + 4) % 26 = 8` $\rightarrow$ `I`
- `L + Y = (11 + 24) % 26 = 9` $\rightarrow$ `J`
- `L + K = (11 + 10) % 26 = 21` $\rightarrow$ `V`
- `O + E = (14 + 4) % 26 = 18` $\rightarrow$ `S`

Zadanie dodatkowe

Deszyfrowanie
Deszyfrowanie w szyfrze Vigenère'a polega na odwróceniu operacji szyfrowania. W szyfrowaniu każdą literę tekstu przesuwaliśmy o wartość wynikającą z odpowiadającej jej litery klucza. Aby odszyfrować wiadomość, zamiast dodawać przesunięcie z klucza, odejmujemy je.
Przykład:
Zaszyfrowana wiadomość: `"RIJVS"`
Klucz: `"KEY"`
Powtórzony klucz: `"KEYKE"`.

- `R - K = (17 - 10 + 26) % 26 = 7` $\rightarrow$ `H`
- `I - E = (8 - 4 + 26) % 26 = 4` $\rightarrow$ `E`
- `J - Y = (9 - 24 + 26) % 26 = 11` $\rightarrow$ `L`
- `V - K = (21 - 10 + 26) % 26 = 11` $\rightarrow$ `L`
- `S - E = (18 - 4 + 26) % 26 = 14` $\rightarrow$ `O`

Wskazówka:
Proszę wykorzystać funkcje `ord()` oraz `chr()`
Kod programu proszę przesłać na adres mailowy jacek.gatlik@agh.edu.pl